

Runtrack C – J03

Job 01

Dans un fichier `is.c`, implémenter les fonctions suivantes. Chacune retourne **1** si une condition est vraie, et **0** sinon. Utiliser uniquement des comparaisons :

`int is_digit(char c);` (c est un chiffre)

`int is_lower(char c);` (c est un caractère minuscule)

`int is_upper(char c);` (c est un caractère majuscule)

`int is_alpha(char c);` (c est alphabétique)

`int is_print(char c);` (c est un caractère imprimable)

Référez-vous éventuellement à la table ASCII pour vérifier les bornes des séries de caractères concernées. **`is_alpha`** devrait réutiliser **`is_lower`** et **`is_upper`**.

Fonctions utilisées : aucune.

Job 02

Dans un fichier `to.c`, implémenter les fonctions suivantes :



```
char to_lower(char c);  
char to_upper(char c);  
char switch_case(char c);
```

Ces fonctions transforment les minuscules en majuscules et inversement. Si le caractère n'est pas alphabétique, il est retourné tel quel. **switch_case** inverse la casse et doit être défini en utilisant les deux autres fonctions.

Fonctions utilisées : aucune.

Job 03

Dans le fichier `is_str.c`, implémenter les fonctions suivantes :

```
int str_is_digit(char *s);  
int str_is_alpha(char *s);  
int str_is_print(char *s);
```

Même rôle que pour le job 01, mais avec des strings. Réutiliser les fonctions du Job 01 (il peut être intéressant de les inclure...).

Fonctions utilisées : celles du job 01 uniquement.

Job 04

Idem que le job 02, mais pour des strings :



```
void str_to_lower(char *s);  
void str_to_upper(char *s);  
void str_switch_case(char *s);
```

Fonctions utilisées : celles du job 02.

Les strings sont modifiées en place. Pensez aussi à réutiliser des fonctions déjà écrites !

Job 05

Dans un fichier `sanitize.c`, implémenter la fonction de nettoyage de chaîne **str_sanitize**, qui remplace tout caractère non imprimable par '?', sans en modifier la longueur :

```
void str_sanitize(char *s);
```

Fonctions autorisées : aucune.

Job 06

Dans un fichier `power.c`, implémentez la fonction **power** qui prend en paramètre deux int et élève le premier à la puissance du deuxième :

```
int power(int n, int power);
```

Fonctions autorisées : aucune.



Job 07

Dans un fichier `factorial.c`, implémenter la fonction **factorial** qui calcule récursivement la factorielle d'un `int` passé en paramètre :

```
int factorial(int n);
```

Fonctions autorisées : aucune.

Job 08

Dans un fichier `fibonacci.c`, implémenter la fonction **fibonacci_it** qui calcule le n-ième élément de la suite de Fibonacci de façon itérative, puis **fibonacci_rec** de façon récursive ($n < 20$) :

```
int fibonacci_it(int n);
```

```
int fibonacci_rec(int n);
```

Fonctions autorisées : aucune.

Job 09

Dans le fichier `my_atoi.c`, implémenter la fonction **my_atoi** qui prend en paramètre une chaîne de caractères et la convertit en `int` :

```
int my_atoi(char *str);
```



Fonctions autorisées : aucune.

Job 10

Dans un fichier `my_itoa.c`, implémenter la fonction **my_itoa** qui prend en paramètre un `int` (positif ou nul) et qui le convertit en chaîne de caractères :

```
char *my_itoa(int n);
```

Fonctions autorisées : `malloc()`

Job 11

Dans un fichier `itoa_hex.c`, implémenter la fonction **itoa_hex** qui prend en paramètre un `int` (positif ou nul) et qui le convertit en hexadécimal, avec les caractères alphabétiques en majuscule :

```
char *itoa_hex(int n);
```

Fonctions autorisées : `malloc()`

Job 12

Dans un fichier `job12.c`, écrivez un programme qui prend un entier `n` en argument et affiche :

- sa valeur en décimal (utilise **my_itoa**)
- sa valeur en hexadécimal (utilise **itoa_hex**)
- sa factorielle si < 10



- le n-ième Fibonacci si $n < 20$

Fonctions autorisées : `write()`, `malloc()`, `free()`

Job 13

Dans un fichier `job13.c`, écrivez un programme qui prend des entiers en arguments, les stocke dans un tableau alloué, affiche leur somme et leur moyenne, et libère toute la mémoire avant de quitter.

Fonctions autorisées : `write()`, `malloc()`, `free()`

```
?> ./job13.exe 123 456 7 8 9
```

603

120.6

Compétences visées

- C

Base de connaissances

- https://www.w3schools.com/c/c_getstarted.php
- <https://openclassrooms.com/fr/courses/19980-apprenez-a-programmer-en-c>
- <https://www.my-mooc.com/fr/mooc/c-programming-getting-started>